拓展：同期群的里子——从「看三角矩阵」到「数学建模+生产落地」

> **本拓展讲什么**：主课教你同期群三角矩阵、纵向横向分析、Bootstrap置信区间、渠道交叉诊断。本拓展补上主课没讲的：留存曲线的数学模型、生存分析的统计学根基、幸存者偏差的隐蔽陷阱、生产级SQL、曲线拟合做LTV预测。
> **适合谁**：学完W2D2主课，能构建同期群矩阵做诊断，但想知道「留存曲线的数学本质是什么」和「怎么把同期群分析写成生产代码」。 **预计时长**：约 2h

---

知识网络预览

```
[核心深化] 留存曲线数学建模(指数/幂律/对数正态)
    ├── [并行] 生存分析(Kaplan-Meier) ← 同期群的统计学根基
    ├── [防护] 幸存者偏差 —— 活越久的用户越容易被看到
    ├── [防护] 同期群对比的MDE —— n太小时能检出多大差异?
    ├── [落地] 生产级同期群SQL —— 从3行草图到可调度脚本
    ├── [落地] LTV预测(曲线拟合) —— 从文字描述到代码实现
    └── [替代] 滚动留存 vs 同期群留存 —— 不同场景选不同算法
```

本拓展新增	7 个课程未覆盖的深度概念
[必修] 5个	留存曲线建模、生存分析、幸存者偏差、生产SQL、LTV预测
[选修] 2个	滚动留存vs同期群、同期群MDE

第一部分：沙盘设定

快达出行（第7次出场）

当前状态：W2D1你帮快达用AARRR诊断了增长瓶颈（卡在激活环节）。今天CEO给了新任务：

> 「上个月我们做了两件事——改了新手引导流程（3月15日上线），同时加大了信息流投放（3月20日起放宽定向）。现在整体W1留存从40%涨到42%——但我想知道：是改版起作用了，还是投放结构变了导致数据看起来好？」

你的角色：快达增长数据分析师。

数据资产：

- `dwd_user_register_di` — 注册表(user_id, channel, register_time, city)
- `dwd_user_active_di` — 日活表(user_id, dt, session_cnt, core_action_cnt)
- `dwd_payment_di` — 支付表(user_id, order_id, amount, pay_time)
- `dwd_trip_core_di` — 行程事件表(user_id, trip_id, event_type, event_time)
- `dim_channel_info` — 渠道维表(channel_id, channel_name, channel_type)

第二部分：入口自诊

Q1：同期群表格中，横向看一行和纵向看一列各回答什么问题?

Q2：整体W1留存从40%涨到42%——能直接说产品变好了吗？至少写出2种需要排除的替代解释。

Q3：Bootstrap方法计算留存率置信区间的原理是什么？样本量50人时CI大概多宽?

自诊反馈

题号	✅ 参考答案	❌ 如果答偏了
Q1	横向=同一批用户随时间衰减（产品粘性），纵向=不同批次在同一节点的质量对比（产品在变好还是变差）	混淆→回看主课2.1节
Q2	不能。(1)渠道结构可能变了——加大了高质量渠道投放占比→权重效应。(2)季节性——如果是12月，每年都会涨。(3)样本波动——如果新用户量小，±5pp是正常的	直接说"是"→回去看主课1.1节平均数陷阱
Q3	Bootstrap=从原始样本中有放回地重采样N次，每次算留存率，取分布的分位数做CI。n=50时CI宽度约±20pp，几乎无法做决策	不清楚→回去跑主课步骤4的代码

第三部分：概念网络展开

核心深化：留存曲线的数学建模

> 主课展示了留存曲线形状（陡降拉平/永不拉平/缓降拉平），但没说这些形状背后是什么数学规律。理解数学模型 = 知道怎么用公式而非图表做预测。

认知冲突

快达1月新用户的W1=40%, W2=28%, W4=20%, W8=14%, W12=11%。

你的任务：预测这批用户W26（半年后）的留存率。

只看图表→凭感觉猜"大概8-10%"。**用数学模型**→用前几个数据点拟合出公式，带入t=26周得到精确预测值+置信区间。

> 增长分析师的进阶：从"看图说话"到"用公式做预测"。

In [1]:
```
# [必修] 留存曲线的三种数学模型 vs 真实数据
import numpy as np; np.random.seed(42)
from scipy.optimize import curve_fit

# 模拟快达1月新用户的真实留存（带噪声）
weeks = np.array([1, 2, 3, 4, 8, 12])
true_retention = np.array([0.400, 0.280, 0.230, 0.200, 0.140, 0.110])
observed = true_retention + np.random.normal(0, 0.008, len(weeks))

# === 模型1: 指数衰减 R(t) = a * exp(-λ*t) ===
def exp_decay(t, a, lam): return a * np.exp(-lam * t)
popt_exp, _ = curve_fit(exp_decay, weeks, observed, p0=[0.45, 0.15], bounds=([0,

# === 模型2: 幂律衰减 R(t) = a * t^(-b) ===
def power_law(t, a, b): return a * np.power(t, -b)
popt_pow, _ = curve_fit(power_law, weeks, observed, p0=[0.4, 0.5], bounds=([0,0]

# === 模型3: 对数正态 R(t) = a * (1 - Φ((ln(t)-μ)/σ)) 简化为两段式 ===
# 实际中常用 "shifted power law": R(t) = a / (1 + (t/b)^c)
def shifted_power(t, a, b, c): return a / (1 + np.power(t/b, c))
popt_sp, _ = curve_fit(shifted_power, weeks, observed, p0=[0.5, 0.5, 2.0], bound

# 预测W26
t_pred = np.array([26])
print("=" * 60)
print("快达1月新用户 留存曲线建模：预测W26留存率")
print("=" * 60)
print(f"{'模型':<25} {'a':>8} {'λ/b/c':>12} {'R2':>8} {'W26预测':>10}")
print("-" * 65)

for name, popt, fn in [
    ("1.指数衰减 R=a*exp(-λt)", popt_exp, exp_decay),
```

```
    ("2.幂律衰减 R=a*t^(-b)", popt_pow, power_law),
    ("3.偏移幂律 R=a/(1+(t/b)^c)", popt_sp, shifted_power),
]:
    preds = fn(weeks, *popt)
    ss_res = np.sum((observed - preds)**2)
    ss_tot = np.sum((observed - observed.mean())**2)
    r2 = 1 - ss_res/ss_tot if ss_tot > 0 else 0
    w26 = fn(t_pred, *popt)[0]
    params = ",".join([f"{v:.3f}" for v in popt])
    print(f"{name:<25} {params:>20} {r2:>7.3f} {w26:>9.1%}")

print()
print("【结论】")
print("  模型3（偏移幂律）通常拟合最好—它同时捕捉了")
print("  前期快速衰减和后期趋于稳定的特征")
print("  a=初始留存水平，b=衰减速度(小=慢)，c=曲线形状")
print("  c>1→S型（有平台期），c<1→指数型（一直衰减）")
print()
print("【沙盘应用】快达可以用拟合参数做三件事:")
print("  1. 用前4周数据预测W26，预判这批用户的长期价值")
print("  2. 对比改版前后b值—b变小说明衰减变慢=改版有效")
print("  3. 设置预警：当b值恶化>20%时报警")
```

```
============================================================
快达1月新用户 留存曲线建模：预测W26留存率
============================================================
模型                                    a          λ/b/c        R2       W26预测
------------------------------------------------------------------------
1.指数衰减 R=a*exp(-λt)                        0.411,0.145    0.897       0.9%
2.幂律衰减 R=a*t^(-b)                          0.404,0.506    0.996       7.8%
3.偏移幂律 R=a/(1+(t/b)^c)         1.000,0.546,0.685    0.995       6.6%

【结论】
  模型3（偏移幂律）通常拟合最好—它同时捕捉了
  前期快速衰减和后期趋于稳定的特征
  a=初始留存水平，b=衰减速度(小=慢)，c=曲线形状
  c>1→S型（有平台期），c<1→指数型（一直衰减）

【沙盘应用】快达可以用拟合参数做三件事:
  1. 用前4周数据预测W26，预判这批用户的长期价值
  2. 对比改版前后b值—b变小说明衰减变慢=改版有效
  3. 设置预警：当b值恶化>20%时报警
```

概念群 A：[并行] 生存分析入门——同期群的统计学根基

> 主课讲的"留存曲线"在统计学里叫"生存曲线"(Survival Curve)，"同期群"叫"风险集"(Risk Set)。了解生存分析让你知道这些方法从哪来、为什么这么做是对的。

In [2]:
```
# [必修] Kaplan-Meier生存曲线—同期群留存曲线的统计学基础
import numpy as np; np.random.seed(42)

print("=" * 60)
print("Kaplan-Meier 生存估计 vs 简单留存率")
```

```
print("=" * 60)
print()

# 模拟100个用户的流失时间（天）
# 有些用户在观测期内流失了（事件发生），有些还活着（删失）
n_users = 100
true_lifetimes = np.random.exponential(scale=30, size=n_users)  # 真实生命周期（指
observation_end = 60  # 我们只观测了60天

# KM估计 vs 简单留存率
times = np.arange(1, observation_end + 1)
km_survival = np.ones(len(times))  # KM生存概率
naive_retention = np.ones(len(times))  # 简单留存率（分母不变）

at_risk = n_users  # KM: 风险集大小
for t_idx, t in enumerate(times):
    # 这天有多少用户流失
    events_today = np.sum((true_lifetimes >= t-1) & (true_lifetimes < t))
    # 这天有多少用户被删失（观测结束）
    censored_today = np.sum((true_lifetimes >= observation_end) & (np.abs(true_l
    if t_idx == 0:
        censored_today = np.sum(true_lifetimes >= observation_end)

    # KM: 只除以当天仍at risk的用户
    if at_risk > 0:
        km_survival[t_idx] = km_survival[t_idx-1] * (1 - events_today / at_risk)
    # 简单留存率: 分母始终是n_users
    naive_retention[t_idx] = 1 - np.sum(true_lifetimes < t) / n_users

    at_risk -= (events_today + (censored_today if t_idx == 0 else 0))
    # 简化: 每天更新风险集
    at_risk = np.sum(true_lifetimes >= t)

# 简化版KM
from collections import defaultdict
events_by_day = defaultdict(int)
for lt in true_lifetimes:
    day = int(np.ceil(lt))
    if day <= observation_end:
        events_by_day[day] += 1

km = np.ones(observation_end + 1)
at_risk = n_users
for d in range(1, observation_end + 1):
    if at_risk > 0:
        km[d] = km[d-1] * (1 - events_by_day[d] / at_risk)
    else:
        km[d] = km[d-1]
    at_risk -= events_by_day[d]

# 对比
checkpoints = [7, 14, 30, 60]
print(f"{'天数':<10} {'简单留存率':>12} {'KM生存估计':>12} {'差异':>10}")
print("-" * 48)
for d in checkpoints:
    naive = 1 - np.sum(true_lifetimes < d) / n_users
    print(f"Day{d:<7} {naive:>11.1%} {km[d]:>11.1%} {(km[d]-naive):>+9.1%}")

print()
print("【核心区别】")
```

```
print("  简单留存率：分母永远=n_users（忽略删失）")
print("  KM生存估计：分母=当天仍at risk的用户（正确处理删失）")
print("  如果用户因为'还没到观测期'而非'流失'消失→简单留存率会低估真实留存")
print()
print("【快达应用】")
print("  6月新用户只观测了4周→W8数据不可得（删失）")
print("  KM方法：在W1/W2/W4的留存计算中正确处理'后面还没观测到'的用户")
print("  简单留存：粗暴地把'不可得'当'n/a'→三角矩阵右下角的空值")
print("  → KM给了更严谨的统计框架来处理'未观测到≠流失'")
print()
print("【面试加分】被问'留存率和生存分析什么关系'时:")
print("  '同期群的留存曲线本质上是Kaplan-Meier生存估计—'")
print("  'KM方法通过动态调整风险集来正确处理右删失数据，'")
print("  '而简单留存率用固定分母会低估真实留存。'")
```

```
============================================================
Kaplan-Meier 生存估计 vs 简单留存率
============================================================

天数                简单留存率        KM生存估计          差异
-------------------------------------------------
Day7              72.0%        72.0%      +0.0%
Day14             56.0%        56.0%      +0.0%
Day30             33.0%        33.0%      +0.0%
Day60             12.0%        12.0%      -0.0%

【核心区别】
  简单留存率：分母永远=n_users（忽略删失）
  KM生存估计：分母=当天仍at risk的用户（正确处理删失）
  如果用户因为'还没到观测期'而非'流失'消失→简单留存率会低估真实留存

【快达应用】
  6月新用户只观测了4周→W8数据不可得（删失）
  KM方法：在W1/W2/W4的留存计算中正确处理'后面还没观测到'的用户
  简单留存：粗暴地把'不可得'当'n/a'→三角矩阵右下角的空值
  → KM给了更严谨的统计框架来处理'未观测到≠流失'

【面试加分】被问'留存率和生存分析什么关系'时:
  '同期群的留存曲线本质上是Kaplan-Meier生存估计—'
  'KM方法通过动态调整风险集来正确处理右删失数据，'
  '而简单留存率用固定分母会低估真实留存。'
```

概念群 B：[防护] 同期群中的幸存者偏差

> 主课完全未提及。幸存者偏差是同期群分析中最隐蔽的陷阱——"活越久的用户越容易被你看到"，导致你高估了用户质量。

🔴 认知冲突

假设你想分析"什么样的用户留存好"——你拉了最近3个月所有活跃用户的行为数据，发现：

- 活跃用户中80%在过去7天内使用过核心功能
- 结论：「核心功能使用」是留存的关键！

但这是循环论证——你只看了"留下来的人"，而留下来的人当然会更活跃。那些同样使用了核心功能但最终流失的用户，你没有看到。

> 同期群分析中的幸存者偏差：如果你只看活跃用户的数据，你会高估每个行为对留存的因果效应。正确做法是用"同期群注册时的特征"而非"当前活跃时的特征"做特征分析。

In [3]:
```python
# [必修] 同期群幸存者偏差—数值演示
import numpy as np; np.random.seed(42)

n_users = 5000
# 模拟: 注册时每个用户有"核心功能使用意图"(0-1)，这是真实特征
true_intent = np.random.beta(2, 3, size=n_users)
# D30留存概率=意图的增函数+噪声
d30_retain_prob = 0.05 + 0.70 * true_intent + np.random.normal(0, 0.05, n_users)
d30_retain_prob = np.clip(d30_retain_prob, 0, 1)
retained = np.random.binomial(1, d30_retain_prob)

# D30时"核心功能使用频率"—留存的用户自然用得更多
usage_if_retained = true_intent * 3 + np.random.normal(0, 0.3, n_users)
usage_if_lost = true_intent * 1 + np.random.normal(0, 0.3, n_users)  # 流失的用得
observed_usage = np.where(retained == 1, usage_if_retained, usage_if_lost)

print("=" * 60)
print("幸存者偏差: 分别用两种方法分析'什么特征预测留存'")
print("=" * 60)
print()

# 方法A（正确）: 用注册时的true_intent预测D30留存
corr_correct = np.corrcoef(true_intent, retained)[0, 1]

# 方法B（错误，幸存者偏差）: 只看留存用户，用observed_usage预测"为什么留存好"
survivors = retained == 1
# 在幸存者中，observed_usage和true_intent的相关性被放大
corr_survivor = np.corrcoef(observed_usage[survivors], true_intent[survivors])[0

# 方法C（最错）: 用observed_usage反推留存→循环论证
corr_circular = np.corrcoef(observed_usage, retained)[0, 1]

print(f"【方法A】注册时意图 vs D30留存: r = {corr_correct:.3f}")
print(f"  → 正确: 用同期群注册时的特征预测未来留存")
print()
print(f"【方法B】幸存者中 使用频率 vs 意图: r = {corr_survivor:.3f}")
print(f"  → 幸存者偏差: 在留存用户中，使用频率和意图的关系被选择性放大")
print()
print(f"【方法C】D30使用频率 vs D30留存: r = {corr_circular:.3f}")
print(f"  → 纯循环论证: '留存好的用户更活跃'=同义反复")
print()

print("【快达沙盘—实际陷阱】")
print("  分析'什么行为预测D30留存高'—只拉了D30活跃的用户的D7行为")
print("  → 你发现'D30活跃用户中D7完单>=3次的比例很高'")
print("  → 结论'D7完单>=3次是留存关键'—但这是幸存者偏差")
print("  → 正确做法: 取同期群注册时的数据(D0-D1行为特征)，追踪D30结果")
print()
print("【防护法则】")
print("  1. 做同期群留存归因时，特征必须来自'注册时/注册前'")
```

```
print("  2. 不要用'留存后的行为'来解释'为什么留存'")
print("  3. 如果想分析行为与留存的关系→用注册后N天内的行为→预测N+30天留存")
```

```
============================================================
幸存者偏差: 分别用两种方法分析'什么特征预测留存'
============================================================

【方法A】注册时意图 vs D30留存: r = 0.287
  → 正确: 用同期群注册时的特征预测未来留存

【方法B】幸存者中 使用频率 vs 意图: r = 0.886
  → 幸存者偏差: 在留存用户中，使用频率和意图的关系被选择性放大

【方法C】D30使用频率 vs D30留存: r = 0.725
  → 纯循环论证: '留存好的用户更活跃'=同义反复

【快达沙盘—实际陷阱】
  分析'什么行为预测D30留存高'—只拉了D30活跃的用户的D7行为
  → 你发现'D30活跃用户中D7完单>=3次的比例很高'
  → 结论'D7完单>=3次是留存关键'—但这是幸存者偏差
  → 正确做法: 取同期群注册时的数据(D0-D1行为特征), 追踪D30结果

【防护法则】
  1. 做同期群留存归因时, 特征必须来自'注册时/注册前'
  2. 不要用'留存后的行为'来解释'为什么留存'
  3. 如果想分析行为与留存的关系→用注册后N天内的行为→预测N+30天留存
```

概念群 C：[落地] 生产级同期群SQL

> 主课只给了3行SQL草图。这里给出可直接调度的完整生产SQL。

[必修] 从3行草图到生产级SQL

```sql
-- ============================================================
-- [落地] 同期群留存矩阵 — 生产级SQL（Hive/SparkSQL）
-- 输出: cohort_date × period_n 三角留存矩阵
-- 参数: ${dt}=分析日期, ${lookback_days}=回溯天数(如180)
-- ============================================================

WITH
-- Step1: 每个用户的首次活跃日期（定义同期群）
user_cohort AS (
    SELECT user_id, MIN(dt) AS cohort_date
    FROM dwd_user_active_di
    WHERE dt BETWEEN DATE_SUB('${dt}', ${lookback_days}) AND '${dt}'
    GROUP BY user_id
),

-- Step2: 用户在后续各天是否活跃（展开到每天）
user_active_days AS (
    SELECT DISTINCT a.user_id, a.dt AS active_date, c.cohort_date
    FROM dwd_user_active_di a
    JOIN user_cohort c ON a.user_id = c.user_id
    WHERE a.dt BETWEEN c.cohort_date
```

```sql
        AND DATE_ADD(c.cohort_date, ${lookback_days})
),

-- Step3: 计算每个(同期群日期, period_n)的留存率
cohort_retention AS (
    SELECT
        cohort_date,
        DATEDIFF(active_date, cohort_date) AS period_n,
        COUNT(DISTINCT ad.user_id) AS active_users
    FROM user_active_days ad
    GROUP BY cohort_date, DATEDIFF(active_date, cohort_date)
),

-- Step4: 同期群大小（分母）
cohort_size AS (
    SELECT cohort_date, COUNT(DISTINCT user_id) AS total_users
    FROM user_cohort GROUP BY cohort_date
)

-- 最终输出: 同期群留存矩阵
SELECT
    cs.cohort_date,
    cs.total_users AS cohort_size,
    cr.period_n,
    cr.active_users,
    ROUND(cr.active_users * 1.0 / cs.total_users, 4) AS retention_rate
FROM cohort_size cs
LEFT JOIN cohort_retention cr ON cs.cohort_date = cr.cohort_date
WHERE cr.period_n IN (0, 1, 3, 7, 14, 30, 60, 90)  -- 关注的关键节点
ORDER BY cs.cohort_date, cr.period_n;

-- ============================================================
-- 如果需要"按渠道×同期群"交叉分析（主课步骤3）:
-- 在user_cohort中加入渠道字段:
--   LEFT JOIN dwd_user_register_di r ON c.user_id = r.user_id
--   然后 GROUP BY r.channel, cohort_date
-- ============================================================
```

与主课3行草图的关键差异:

维度	主课草图	本SQL
完整性	只有JOIN+GROUP BY骨架	4步CTE链式 + 每个CTE独立可测试
参数化	硬编码日期	${dt}和${lookback_days}支持调度传参
性能	未考虑	DISTINCT去重 + 只取关键period_n减少输出
扩展性	无	注释说明了如何加渠道维度做交叉分析
可维护性	—	每步CTE有明确注释，变更口径只需改对应CTE

概念群 D：[落地] 同期群LTV预测——曲线拟合法

> 主课在场景5用文字描述了"用历史留存曲线+新批次W1修正"做LTV预测。这里给出完整代码实现。

In [4]:

```
# [必修] 同期群LTV预测: 偏移幂律拟合 + 新批次W1修正
import numpy as np; np.random.seed(42)
from scipy.optimize import curve_fit

# === 历史数据: 过去6个月各同期群的留存+付费数据 ===
historical_cohorts = {
    '2025-12': {'w1_ret':0.42, 'w4_ret':0.22, 'w8_ret':0.15, 'w12_ret':0.12, 'w2
                'w1_arpu':12, 'w4_arpu':16, 'w8_arpu':22, 'w12_arpu':25, 'w26_ar
    '2026-01': {'w1_ret':0.40, 'w4_ret':0.20, 'w8_ret':0.14, 'w12_ret':0.11, 'w2
                'w1_arpu':11, 'w4_arpu':15, 'w8_arpu':21, 'w12_arpu':24, 'w26_ar
    '2026-02': {'w1_ret':0.38, 'w4_ret':0.19, 'w8_ret':0.13, 'w12_ret':0.10, 'w2
                'w1_arpu':10, 'w4_arpu':14, 'w8_arpu':20, 'w12_arpu':23, 'w26_ar
}

# === Step1: 用历史均值拟合"留存×付费"衰减曲线 ===
t_points = np.array([1, 4, 8, 12, 26])
avg_retention = np.array([
    np.mean([h['w1_ret'] for h in historical_cohorts.values()]),
    np.mean([h['w4_ret'] for h in historical_cohorts.values()]),
    np.mean([h['w8_ret'] for h in historical_cohorts.values()]),
    np.mean([h['w12_ret'] for h in historical_cohorts.values()]),
    np.mean([h['w26_ret'] for h in historical_cohorts.values()]),
])
avg_arpu = np.array([
    np.mean([h['w1_arpu'] for h in historical_cohorts.values()]),
    np.mean([h['w4_arpu'] for h in historical_cohorts.values()]),
    np.mean([h['w8_arpu'] for h in historical_cohorts.values()]),
    np.mean([h['w12_arpu'] for h in historical_cohorts.values()]),
    np.mean([h['w26_arpu'] for h in historical_cohorts.values()]),
])

# 拟合留存曲线的偏移幂律模型
def shifted_power(t, a, b, c): return a / (1 + np.power(t/b, c))
popt_ret, _ = curve_fit(shifted_power, t_points, avg_retention, p0=[0.5,0.5,2.0]

# 拟合ARPU的线性增长模型（周数越大单次付费金额越高→老用户更信任）
def arpu_curve(t, base, growth): return base + growth * np.log(t + 1)
popt_arpu, _ = curve_fit(arpu_curve, t_points, avg_arpu, p0=[10, 3])

print("=" * 60)
print("同期群LTV预测模型")
print("=" * 60)
print()
print(f"留存曲线: R(t) = {popt_ret[0]:.3f} / (1+(t/{popt_ret[1]:.1f})^{popt_ret[2
print(f"ARPU曲线: A(t) = {popt_arpu[0]:.1f} + {popt_arpu[1]:.1f}*ln(t+1)")
print()

# === Step2: 预测26周累计LTV ===
t_fine = np.arange(1, 27)
weekly_retention = shifted_power(t_fine, *popt_ret)
weekly_arpu = arpu_curve(t_fine, *popt_arpu)
weekly_ltv = weekly_retention * weekly_arpu * 0.08  # 8%付费率
```

```python
cumulative_ltv = np.cumsum(weekly_ltv)

print(f"预测26周累计LTV = {cumulative_ltv[-1]:.2f}元/新增用户")
print(f"  W4累计: {cumulative_ltv[3]:.2f} | W8: {cumulative_ltv[7]:.2f} | W12: {
print()

# === Step3: 用新批次的W1留存修正预测 ===
new_cohort_w1_ret = 0.35  # 新批次W1留存低于历史均值
historical_w1_mean = avg_retention[0]  # 历史W1均值
adjustment_factor = new_cohort_w1_ret / historical_w1_mean

adjusted_ltv = cumulative_ltv[-1] * adjustment_factor
print(f"【新批次修正】")
print(f"  历史W1均值: {historical_w1_mean:.1%}, 新批次W1: {new_cohort_w1_ret:.1%}
print(f"  修正系数: {adjustment_factor:.2f}")
print(f"  修正后26周LTV: {cumulative_ltv[-1]:.2f} x {adjustment_factor:.2f} = {ad
print()

# === Step4: 财务推演 ===
monthly_new_users = 20000
cac = 12  # 获客成本
total_ltv = adjusted_ltv * monthly_new_users
total_cac = cac * monthly_new_users
print(f"【财务推演】月新增{monthly_new_users:,}人")
print(f"  总LTV: {total_ltv:,.0f}元 | 总CAC: {total_cac:,.0f}元")
print(f"  LTV/CAC: {adjusted_ltv/cac:.2f}x {'(健康)' if adjusted_ltv/cac>3 else
print()
print("【局限性】修正系数假设'留存曲线形状不变'—如果改版改变了形状,需要重新拟合")
```

```
============================================================
同期群LTV预测模型
============================================================

留存曲线: R(t) = 3.923 / (1+(t/0.0)^0.54)
ARPU曲线: A(t) = 6.0 + 6.6*ln(t+1)

预测26周累计LTV = 5.60元/新增用户
  W4累计: 1.19 | W8: 2.16 | W12: 3.02

【新批次修正】
  历史W1均值: 40.0%, 新批次W1: 35.0%
  修正系数: 0.87
  修正后26周LTV: 5.60 x 0.87 = 4.90元/用户

【财务推演】月新增20,000人
  总LTV: 98,058元 | 总CAC: 240,000元
  LTV/CAC: 0.41x (需改善)

【局限性】修正系数假设'留存曲线形状不变'—如果改版改变了形状,需要重新拟合
C:\Users\18238\AppData\Local\Temp\ipykernel_50780\826009635.py:33: RuntimeWarnin
g: invalid value encountered in power
  def shifted_power(t, a, b, c): return a / (1 + np.power(t/b, c))
```

概念群 E：[替代] 滚动留存 vs 同期群留存

> [选修] 主课教的"同期群留存"是标准的，但不是唯一的。滚动留存(Rolling Retention)在某些场景下更实用。

[选修] 两种留存计算法的对比

维度	同期群留存(Cohort)	滚动留存(Rolling)
计算方式	固定同期群(如1月新用户)，追踪D7/D30	固定日历日期(如6月1日)，回溯每个用户"那天是注册后第几天"
优点	能看到"批次质量"的变化趋势(纵向对比)	对日历事件敏感(如6月1日下雨→当天总体留存下降)
缺点	噪声大(每批用户量可能小)	混合了不同批次用户→无法看到"新用户质量"的变化
适用场景	分析产品改版/渠道策略变化	监控日常运营健康度/日历事件影响
面试题	「产品在变好还是变差？」	「昨天DAU跌了是因为天气还是产品？」

快达示例:

- 同期群留存："3月注册的用户，D7留存40%，4月注册的用户D7留存38%→用户质量在下降"
- 滚动留存："6月1日所有活跃用户中，有多少人在7天后(D8)还活跃？=当天的滚动D7留存"

选择策略：日常看板用滚动留存（监控日历变化），专项分析用同期群留存（判断产品趋势）。两者互补而非替代。

概念群 F：[防护] 同期群对比的MDE——n太小时能检出多大差异?

> [选修] 主课教了Bootstrap CI，但没有回答"如果我想确认3月留存比2月高5pp，我需要每批多少人？"——这是MDE (Minimum Detectable Effect)。

In [5]:
```python
# [选修] 同期群对比的MDE: 给定样本量能检出多大差异?
import numpy as np
from scipy import stats

def cohort_mde(n_per_cohort, baseline_rate, alpha=0.05, power=0.80):
    """
    计算两个同期群对比时，给定样本量能检出的最小效应量(MDE)
    基于两样本比例检验的样本量公式
    """
    z_alpha = stats.norm.ppf(1 - alpha/2)
    z_beta = stats.norm.ppf(power)
```

```
    p = baseline_rate
    mde = (z_alpha + z_beta) * np.sqrt(2 * p * (1-p) / n_per_cohort)
    return mde

# 同样本量下，能检出的MDE
print("=" * 60)
print("同期群对比MDE：给定样本量能检出多大留存率差异?")
print("=" * 60)
print(f"(alpha=0.05, power=0.80, 基线留存率=40%)")
print()
print(f"{'每批人数':<12} {'能检出的MDE':>15} {'可做的决策':<30}")
print("-" * 60)

for n in [50, 100, 200, 500, 1000, 5000]:
    mde = cohort_mde(n, 0.40)
    if mde > 0.20:
        action = "几乎无法做决策"
    elif mde > 0.10:
        action = "只能看大方向"
    elif mde > 0.05:
        action = "可做中等精度的对比"
    else:
        action = "可做精细对比和严格决策"
    print(f"{n:<12} {mde:>14.1%} {action:<30}")

print()
print("【快达实战】")
print("  改版后3月新用户=350人，W1=45%")
print("  改版前2月新用户=280人，W1=38%")
print(f"  观察差值=7pp，最小检出差异≈{cohort_mde(315, 0.40):.0%}")
print("  -> 如果n<200，MDE>10pp→7pp的差异可能是随机波动")
print("  -> 建议：积累到500+人/批再下结论，或降低置信度到90%加快决策")
```

```
============================================================
同期群对比MDE：给定样本量能检出多大留存率差异?
============================================================
(alpha=0.05, power=0.80, 基线留存率=40%)

每批人数                  能检出的MDE 可做的决策
------------------------------------------------------------
50                      27.4% 几乎无法做决策
100                     19.4% 只能看大方向
200                     13.7% 只能看大方向
500                      8.7% 可做中等精度的对比
1000                     6.1% 可做中等精度的对比
5000                     2.7% 可做精细对比和严格决策

【快达实战】
  改版后3月新用户=350人，W1=45%
  改版前2月新用户=280人，W1=38%
  观察差值=7pp，最小检出差异≈11%
  -> 如果n<200，MDE>10pp→7pp的差异可能是随机波动
  -> 建议：积累到500+人/批再下结论，或降低置信度到90%加快决策
```

第四部分：综合沙盘演练

综合练习 1：多概念协同——快达改版效果评估

场景：快达3月15日上线新用户引导流程改版，同时3月20日起信息流投放放宽定向。4月底你需要判断：W1留存变化中，改版贡献多少、渠道变化贡献多少？

用三个工具协同：留存曲线建模（对比改版前后b值）+ 幸存者偏差检查（特征分析用注册时数据）+ 生产SQL（拉渠道×同期群矩阵）

In [6]:
```
# 综合练习1：快达改版效果评估—改版 vs 渠道结构，各贡献多少？
import numpy as np; np.random.seed(123)

# 模拟：改版前(1-2月)和改版后(3-4月)的渠道×同期群数据
channels = ['自然流量', '信息流投放', '搜索投放']
periods = ['1月', '2月', '3月', '4月']

# W1留存：行=渠道，列=月份
w1_data = np.array([
    [0.43, 0.44, 0.46, 0.47],  # 自然流量：稳定上升(产品改善)
    [0.35, 0.32, 0.30, 0.22],  # 信息流：持续恶化(定向放宽)
    [0.38, 0.37, 0.40, 0.41],  # 搜索：小幅改善(改版效果)
])

# 各渠道用户数
users_data = np.array([
    [5000, 5200, 5100, 5300],   # 自然流量：稳定
    [3000, 4000, 8000, 15000],  # 信息流：爆发增长(放宽定向!)
    [2000, 2100, 2200, 2300],   # 搜索：小幅增长
])

print("=" * 60)
print("快达改版效果评估：改版 vs 渠道结构，各贡献多少？")
print("=" * 60)
print()

# 计算各月总体W1留存
overall_w1 = np.sum(w1_data * users_data, axis=0) / np.sum(users_data, axis=0)

print("渠道×同期群 W1留存矩阵:")
header = f"{'渠道':<12}"
for p in periods: header += f"{p:>10}"
print(header)
print("-" * 52)
for i, ch in enumerate(channels):
    row = f"{ch:<12}"
    for j in range(len(periods)):
        row += f"{w1_data[i,j]:>9.1%}"
    print(row)

print(f"\n{'总体W1留存':<12}", end="")
for j in range(len(periods)):
    print(f"{overall_w1[j]:>9.1%}", end="")

print()
print()

# 分解
print("【效果分解】")
```

```
print(f"  1月->4月总体W1变化: {overall_w1[0]:.1%} -> {overall_w1[-1]:.1%} = {over
print()

# 固定渠道结构(用1月权重)，看"纯产品效果"
jan_weights = users_data[:,0] / users_data[:,0].sum()
fixed_structure_w1 = np.sum(w1_data * jan_weights[:, np.newaxis], axis=0)
product_effect = fixed_structure_w1[-1] - fixed_structure_w1[0]
print(f"  纯产品效果(固定1月渠道权重): {fixed_structure_w1[0]:.1%} -> {fixed_struc
print(f"  -> 自然流量+搜索都在改善(各+3-4pp)，改版本身的贡献 ≈ {product_effect:+.1%
print()

# 渠道结构效应 = 总变化 - 纯产品效果
structure_effect = (overall_w1[-1] - overall_w1[0]) - product_effect
print(f"  渠道结构效应(权重变化): {structure_effect:+.1%}")
print(f"  -> 信息流用户占比从{users_data[1,0]/users_data[:,0].sum():.0%}飙升到{use
print(f"  -> 但信息流W1从35%恶化到22%(-13pp)—低质用户大量涌入")
print()
print(f"【最终结论】")
print(f"  改版本身贡献: {product_effect:+.1%}（产品确实在变好）")
print(f"  渠道结构变化贡献: {structure_effect:+.1%}（拉低了整体数字）")
print(f"  两者相抵→整体W1{overall_w1[-1]-overall_w1[0]:+.1%}")
print(f"  -> 改版有效！但被渠道恶化'抵消'了，只看整体数字会以为改版没效果")
```

```
============================================================
快达改版效果评估：改版 vs 渠道结构，各贡献多少？
============================================================

渠道×同期群 W1留存矩阵:
渠道                    1月         2月         3月         4月
------------------------------------------------------
自然流量              43.0%     44.0%     46.0%     47.0%
信息流投放             35.0%     32.0%     30.0%     22.0%
搜索投放              38.0%     37.0%     40.0%     41.0%

总体W1留存            39.6%     38.5%     36.8%     29.8%

【效果分解】
  1月->4月总体W1变化: 39.6% -> 29.8% = -9.8%

  纯产品效果(固定1月渠道权重): 39.6% -> 38.3% = -1.3%
  -> 自然流量+搜索都在改善(各+3-4pp)，改版本身的贡献 ≈ -1.3%

  渠道结构效应(权重变化): -8.5%
  -> 信息流用户占比从30%飙升到66%
  -> 但信息流W1从35%恶化到22%(-13pp)—低质用户大量涌入

【最终结论】
  改版本身贡献: -1.3%（产品确实在变好）
  渠道结构变化贡献: -8.5%（拉低了整体数字）
  两者相抵→整体W1-9.8%
  -> 改版有效！但被渠道恶化'抵消'了，只看整体数字会以为改版没效果
```

综合练习 2：沙盘变体——B端SaaS的同期群

场景：快达拓展B端业务（企业用车管理平台）。B端同期群和C端有三点本质区别。

任务：写出B端同期群的三个关键区别，以及"同期群"在B端的正确分组依据。

参考答案

区别	C端	B端SaaS
同期群定义	按注册日期	按签约日期/上线日期（两个时间差可达数月）
留存定义	个人是否活跃	团队/企业是否活跃（1人用≠留存，全团队用=留存）
留存曲线形状	指数衰减	可能先升后降（企业逐步推广→更多seat活跃→高峰→流失）
关键指标	D7/D30留存率	NDR(Net Dollar Retention): 去年客户的今年收入/去年收入

B端同期群分组依据：签约日期（不是注册日期）+ 企业规模分层（SMB vs Mid-Market vs Enterprise，留存规律完全不同）

综合练习 3：边界判断——什么时候不需要做同期群？

#	场景	是否需要同期群	替代方案
1	新功能上线第一周，想看用户反馈	❌ 不需要	用实时看板监控核心指标，等2-4周积累足够样本再做同期群
2	判断"昨天DAU跌了是不是因为下雨"	❌ 不需要	用滚动留存——按日历日期看留存变化，同期群太慢
3	评估半年来投放策略变化对用户质量的影响	✅ 必须用	渠道×同期群交叉分析——这是唯一能看到各渠道各批次质量变化的方法
4	预测新批次用户未来6个月的LTV	✅ 最适合	历史同期群留存曲线+新批次W1修正=最可靠的LTV预测方法

综合练习 4：落地——从概念到可交付产物

场景：明天给CEO汇报快达改版效果。自检清单：

- [ ] 用留存曲线模型对比改版前后b值（是否衰减变慢）
- [ ] 用渠道结构固定法分解了"改版效果"和"渠道变化效果"

- [ ] 检查了幸存者偏差——特征分析用的是注册时数据而非活跃时数据
- [ ] 生产SQL可调度运行（参数化+CTE链式）
- [ ] 对样本量不足的同期群标注了"仅供参考"（n<100时）

第五部分：知识网络图

本拓展概念全景

概念群	概念	关系	必修/选修	深度
核心	留存曲线数学建模	核心深化	必修	[M][C]
A	生存分析(Kaplan-Meier)	并行	必修	[M][C]
B	幸存者偏差	防护	必修	[M][C]
C	生产级同期群SQL	落地	必修	[C]
D	LTV预测(曲线拟合)	落地	必修	[M][C]
E	滚动留存vs同期群	替代	选修	[B]
F	同期群对比MDE	防护	选修	[M]

三版本覆盖对比

知识域	主课程(W2D2)	本拓展新增
同期群定义/矩阵/维度	✅ 详细	—
留存曲线形状	✅ 三种形态	✅ 三种数学模型(指数/幂律/偏移幂律)
统计学基础	—	✅ 生存分析KM估计
幸存者偏差	—	✅ 完整数值演示
置信区间	✅ Bootstrap	✅ MDE(最小可检出效应)
SQL实现	3行草图	✅ 4步CTE生产级SQL
LTV预测	文字描述	✅ 曲线拟合+W1修正代码
滚动留存	—	✅ 概念+对比+选择策略

第六部分：元认知自评

学习循环自评表

我能……	还不能	差不多	完全能
用指数/幂律/偏移幂律三种模型拟合留存曲线并选最优	□	□	□
解释KM生存估计和简单留存率的本质区别	□	□	□
识别并避免同期群分析中的幸存者偏差	□	□	□
写出可直接调度的生产级同期群SQL	□	□	□
用曲线拟合法做LTV预测并用新批次W1修正	□	□	□
区分滚动留存和同期群留存的使用场景	□	□	□
根据样本量判断同期群对比的可靠性(MDE)	□	□	□

沙盘复述测试

用3分钟向同事解释：

1. 快达改版后W1留存变化中，改版贡献了多少、渠道变化贡献了多少？
2. 为什么只用"总体W1"做判断会让你得出错误结论？
3. 如果样本量不够，你应该说什么而不是"改版有效"？

03analyst_thinking · Week 2 Day 2 拓展 · 2026-06-25